

Improved YOLOv8-Based Monocular Visual Recognition Method for Pallets

Huijuan Zhang¹, Guoyun Ye², Feng Tian², Zhehang Qiu¹, Jie Zhou^{1,*}

¹ NingboTech University, Ningbo Zhejiang, 315100, China

² Ningbo Ruyi Joint Stock Co., Ltd, Ningbo Zhejiang 315600, China

*Corresponding author's email: zhoujie@nbt.edu.cn

Abstract—Pallet and forklift coordination and handling is an important part of automated warehouse environments. Addressing the issue of pallet recognition failure caused by interference from various factors in complex warehouse scenarios for unmanned forklift AGVs (Autonomous Guided Vehicles), a pallet recognition method based on improved YOLOv8-Pose is proposed in this paper. Firstly, part of C2f (channel-to-pixel) is replaced with C2f_DCN (Deformable convolution network, DCN) to adapt to geometric changes of pallets. The SEAttention module is added to reduce the influence of irrelevant features and improve network performance. And the SIoU loss considering pallet angle information is introduced to improve training accuracy. Secondly, based on obtaining target box and keypoint information, Canny edge detection is utilized to adaptively adjust edge features to obtain more accurate contour information. Through geometric constraint analysis, the calculation relationship between the tilt angle and distance of the pallet relative to the forklift is obtained. Finally, a self-made pallet dataset is constructed to verify the algorithm through the dataset and actual environment. Experimental results show that the proposed method is superior to the unmodified YOLOv8s algorithm in both target detection and keypoint detection, achieving an average precision of 93.3%, a recall rate of 98.3%, and an average keypoint accuracy improvement of 5.7%. This method is applicable for pallet recognition of unmanned forklift AGVs.

Keywords: Visual Recognition; Improved YOLOv8-Pose; Pallet Detection

I. INTRODUCTION

Wheeled robots are widely used in industrial warehousing to achieve intelligent handling of containers. Among them, the unmanned forklift AGVs with pallets have strong adaptability, simple operation, and are widely used. Pallet recognition technology, as the key to unmanned industrial vehicles, is of great significance in improving handling efficiency. Pallet detection and positioning refers to obtaining various forms of data through various sensors, identifying pallets through specific processing methods, and further calculating pallet pose. Currently, how to improve the accuracy and speed of pallet recognition has become an urgent challenge to be overcome.

In warehousing, pallet recognition provides an important foundation for trajectory tracking. By accurately identifying the pose of the pallet, the system can calculate the relative relationship between the pallet and the forklift in real time, ensuring that the forklift can track the trajectory of the pallet relative to the forklift. Therefore, the unmanned forklift AGV can accurately dock with the pallet and complete the handling task. At present, the existing pallet recognition methods are

mainly divided into three categories: sensor-based methods, depth data-based methods and deep learning-based methods.

The sensor-based methods mainly achieve pallet pose through various labels and chips. For example, Lecking et al.[1] proposed a label-based pallet recognition method that calculated pose by scanning reflective plates. Minho et al.[2] combined wireless network and RFID (Radio Frequency Identification) technology to install RFID chips on pallets, achieving warehouse pallet positioning and overall management. Some studies[3] have proposed a pallet positioning method based on AprilTag tags. It has high accuracy under specific conditions and low overall computational complexity. However, due to high hardware costs such as tags and chips, the flexibility and robustness of these methods are poor, and practical applications are limited.

The depth data-based methods mainly rely on devices such as Lidar and structured light cameras. Mohamad et al.[4] used laser to collect data and combined it with machine learning to achieve pallet pose calculation. The detection success rate was high and it can adapt to multi-pallet environments. Walter et al.[5] used nearest edge detection by Lidar to achieve pallet recognition, which can meet the accuracy requirements of engineering. However, the cost of laser equipment cannot be borne by small warehouses. In addition, in complex environments, it is difficult to filter pallet information from 2D laser data, which limits its recognition accuracy in complex environments. Wang et al.[6] achieved relatively efficient forklift automatic positioning and pallet picking using structured light cameras by optimizing sensor design and visual servo control. Xiao et al.[7] used RGB-D cameras to capture depth images. Image segmentation through region growing was implemented, and heuristic filtering methods to remove irrelevant parts was used, followed by template matching. However, the hardware is complex and lacks practicality. The above methods generally have high accuracy, but the hardware cost is high and the speed is slow.

Deep learning-based methods, visual sensors are usually directly used. In recent years, due to fast speed, high accuracy and strong adaptability, a large number of scholars have applied deep learning methods to pallet recognition tasks. Some directly use existing algorithms to achieve pallet recognition. Chen[8] used YOLOv4 combined with single target tracking and pallet feature extraction to achieve continuous pallet tracking and detection. Ye[9] combined YOLOv4 with GrabCut and center positioning, and used post-processing to improve accuracy. To further improve the efficiency of pallet recognition and cope with complex warehousing situations, many studies have

improved existing object detection methods. Zhu et al.[10] introduced transfer learning and improved the keypoint detection algorithm based on the CenterNet algorithm, significantly improving the accuracy of pallet recognition and horizontal angle estimation. However, this method cannot calculate the specific pose of the pallet. Efthimios et al.[11] implemented pallet column detection based on YOLOv5, and used this data to assist point cloud segmentation to achieve pallet recognition. However, this method requires a large amount of computation. Liu[12] also based on the YOLOv5, mainly improved from the perspective of lightweight, and designed a network that can be deployed in embedded NPUs through an adaptive focus detection framework, multi-scale fusion, and lightweight convolution. However, the robustness of the pallet pose calculation method is poor. Based on the YOLOv7 algorithm, Zhou[13] introduced attitude detection through transfer learning to achieve the detection of key corner points and grouping of pallets. At the same time, the loss function was improved by introducing more suitable loss functions SIOU and WingLoss for pallet recognition, which improved detection efficiency. However, in actual storage environments, the success rate of pallet recognition using the above methods is insufficient, especially for images with severe pallet deformation where the accuracy decreases significantly. In addition, none of the above methods have designed a pallet pose calculation method for situations where the keypoints of the pallet are incomplete, resulting in failure when the distance is close. There is still improvement in accuracy and speed of existing methods.

A monocular pallet recognition algorithm based on improved YOLOv8-Pose is proposed in this paper. Firstly, improve the algorithm by replacing some C2f with C2f_DCN to adapt to the significant geometric changes of the pallet; add SEAttention module to reduce the impact of irrelevant features and improve network performance; and introduce SIOU loss considering pallet angle information to improve training accuracy. Secondly, based on obtaining the target box and keypoint information, Canny edge detection is used to adaptively adjust edge features and obtain more accurate contour information. Obtain the calculation relationship between the inclination angle and distance of the pallet relative to the forklift. Finally, a self-made pallet dataset containing two sub datasets, namely the pallet pose validation dataset and the missing keypoint dataset, were used to validate the proposed method. The experiments are conducted to demonstrate the efficiency of the algorithm.

The structure of this paper is as follows: part I is the introduction. Part II is an introduction to the YOLOv8-Pose algorithm. The description of the improved YOLOv8-Pose method is part III. Part IV if Experimental dataset and evaluation metrix. Part V is the experimental results and analysis and discussions. The last part is the conclusions.

II. YOLOV8-POSE ALGORITHM INTRODUCTION

YOLO(You Only Look Once)[14] is a deep learning framework that focuses on real-time object detection. As one of the most widely used versions in the YOLO series, YOLOv8 aims to improve detection accuracy and processing speed. The model inherits the advantages of previous generations and undergoes multiple optimization improvements, including

designing a more complex network structure, a more efficient training process, and stronger feature extraction capabilities. YOLOv8 provides five different basic models for users with different needs, namely YOLOv8n, YOLOv8s, YOLOv8m, YOLOv8l, and YOLOv8x. The parameter quantity of the n/s/m/l/x models increases in order, and the accuracy also increases accordingly.

During the docking process between the unmanned forklift AGV and the pallet, the proportion of the pallet in the image gradually increases as the distance decreases. To balance the recognition accuracy and recognition speed of large and small targets, the scaling factor of YOLOv8s is appropriate. In the YOLOv8 model, a pose detection model based on YOLO-Pose[15] is provided. Compared to the YOLOv8 model, it is improved in the detection head layer to achieve keypoint detection.

The overall structure of YOLOv8-Pose consists of three parts: Backbone(main network), Neck(neck network) and Head(output network). The network structure diagram is shown in Figure 1.

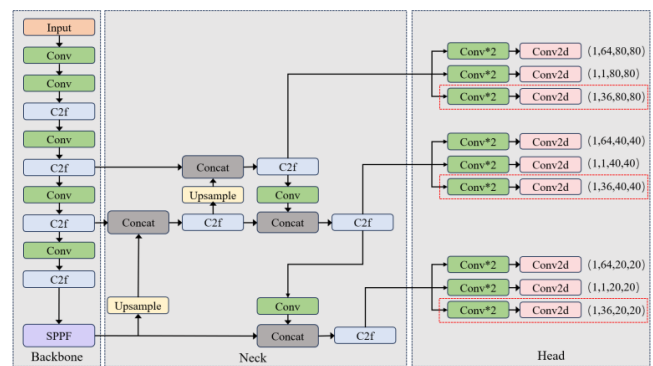


Figure 1. YOLOv8-Pose network architecture

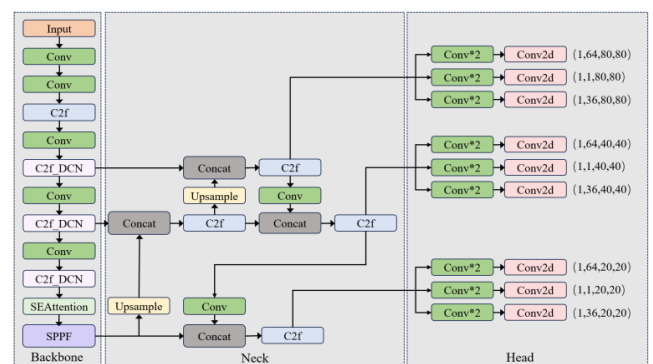


Figure 2. The improved YOLOv8-Pose network architecture

III. IMPROVEMENT METHOD OF YOLOV8-POSE

A. C2F_DCN module

The improved YOLOv8-Pose network architecture is shown Figure 2. In the YOLOv8-Pose backbone network, the C2f module is mainly used to extract features, but due to the fixed nature of the convolution kernel, the receptive field is limited, often focusing only on local information. In the task of picking up items from a pallet, the size and angle of the pallet are constantly changing, and the fixed receptive field of the C2f

module may lead to missed detections. DCN has a more flexible receptive field, and it can effectively increase the robustness of the network[16]. Therefore, by introducing deformable convolution into the C2f module, the network's feature extraction ability can be effectively strengthened, resulting in better performance in pallet recognition tasks.

In conventional convolution methods, the convolution kernel usually has a fixed size and position. Deformable convolution adds a prescribed offset to each pixel position in the convolution kernel, making it more adaptable to geometric changes in objects. In network convolution computation, suppose a set of sampled spatial matrices R is obtained from the input features, with the following form:

$$R = \{(-1,1), (-1,0), (-1,-1), (0,-1), (0,0), (0,1), (-1,-1), (-1,0), (-1,1)\} \quad (1)$$

For the center sampling $V_0\{v_n|n = 1,2, \dots, N\}$ point, it is defined as the sampling point position, and the conventional convolution calculation formula is as follows:

$$y(v_0) = \sum_{p_n \in R} w(v_n) \cdot x(v_0 + v) \quad (2)$$

Where, $w(v_n)$ represents the v_n weight of the weighted calculation unit, x is the input feature map, and y is the output feature map. The DCN introduces an offset $\{\Delta v_n|n = 1,2, \dots, N\}$, and for any position v , the output feature map is expressed as follows:

$$y(s_0) = \sum_{p_0 \in R} w(v_n) \cdot x(v_0 + v_n + \Delta v_n) \quad (3)$$

Considering that the offset is usually a fraction, it needs to be processed by double linear interpolation and integer processing, as shown in Equation (4) ~ Equation(6):

$$X(v) = \sum_q g(q, v) \cdot g(q_y, v_y) \quad (4)$$

$$G(q, v) = g(q_x, v_x) \cdot g(q_y, v_y) \quad (5)$$

$$g(q_y, v_y) = \max(0, 1 - |a - b|) \quad (6)$$

Where, q represents the position of the weight on the feature map, $g(\cdot, \cdot)$ represents the bilinear interpolation convolution kernel, and this interpolation method calculates the target pixel of the interpolation through four points of the original image. q_x and q_y are the original ordinates of integer points, and v_x and v_y are the offset ordinates.

Specifically, deformable convolution first performs pre-computation on input feature maps. For pixels with regular distributions, it generates offset and modulation values through bilinear interpolation. The convolution kernel then utilizes these offsets to adjust sampling point positions, enabling sampling at deformed locations within the input feature map. Subsequently, element-wise multiplication between the offset-processed kernel and the original feature map yields the final convolution result. Notably, the offsets are dynamically adjusted during network training through learning mechanisms, ensuring effective performance.

There are four C2f modules in the YOLOv8-Pose backbone network, among which three C2f modules are feature connected to the neck network. The C2f_DCN module introduces offset to

expand the receptive field during feature extraction, thereby improving the feature extraction capability of the backbone network. In the field of drone target detection, where the task involves detecting small objects with multi-scale and complex backgrounds, Shen[17] embedded three C2F_DCN modules in the YOLOv8 backbone network to enhance the adaptability of the network to varying target shapes. The results demonstrate that replacing part of the C2F modules in the YOLOv8n backbone network with C2F_DCN improves the mAP@0.5 by 0.8%. This paper draws on the proposed method in the aforementioned paper[17]. Considering the feature fusion structure applied to the neck, direct output can preserve the fused feature information to a greater extent. The three feature maps of different scales output by the backbone network are connected through upsampling. Therefore, three C2F_DCN modules are used in Backbone to replace the C2F module, namely the 2nd to 4th C2F modules, which are used for the 4th, 6th, and 8th layers of the backbone feature extraction network, respectively. The structure of C2f_DCN module is shown in Figure 3. The model's receptive field was expanded while minimizing training complexity.

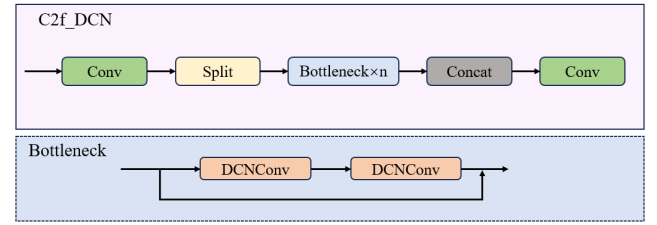


Figure 3. Structure of C2f_DCN module

B. SEAttention Module

In the field of deep learning, introducing attention mechanisms is a common approach to enhance the performance of network model. Attention modules are generally characterized by their versatility, enabling algorithmic improvements from spatial, frequency domain, or channel perspectives with minimal computational overhead. As a channel attention module, SEAttention assigns different weights to channels based on varying degrees of feature relevance to the target. By leveraging trainable weights, it reduces interference from irrelevant features while enhancing the influence of channels aligned with the training objective[18]. This study incorporates this module into the YOLOv8 network to improve algorithm performance.

The steps of SEAttention can be divided into three steps: compression, excitation and output. The overall steps of the module are shown in Figure 4.

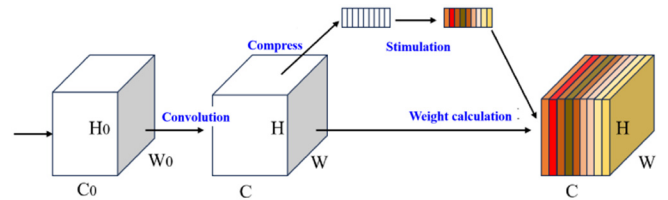


Figure 4. Compression and excitation module structure

In the compression step, the global average pooling method is applied to compress the input matrix into a vector. This vector

is then mapped to a single numerical value through a fully connected layer, ultimately $\mathbf{X} \in \mathbb{R}^{H \times W \times C}$ yielding global feature values for each channel. Specifically, for a given input feature map (where H denotes height, W represents width, and C indicates the number of channels), the calculation is performed using Equation (3)~Equation(7):

$$\mathbf{z}_c = \frac{1}{H \times W} \sum_{i=1}^H \sum_{j=1}^W \mathbf{X}_c(i, j) \quad (7)$$

Where, $\mathbf{z}_c$ represents the global average pooling value of the c -th channel, and $\mathbf{X}_c(i, j)$ represents the value of the input feature matrix at the c -th channel (i, j) .

In the excitation step, two connection layers are first used to adjust the channel number of the global information feature vector, and then the nonlinear change is carried out by using the specified activation function. Finally, the vector elements are normalized and compressed into $[0, 1]$, and the weight values are obtained. The formula is as follows:

$$\mathbf{s} = \sigma(\mathbf{W}_2 \cdot \text{ReLU}(\mathbf{W}_1 \cdot \mathbf{z})) \quad (8)$$

Where, $\mathbf{s}$ is the final weight vector; $\mathbf{W}_1 \in \mathbb{R}^{\frac{C}{r} \times C}$ is the weight matrix of the first fully connected layer, which reduces the number of channels to $\frac{C}{r}$; ReLU is the activation function; $\mathbf{W}_2 \in \mathbb{R}^{\frac{C}{r} \times C}$ is the weight matrix of the second fully connected layer, which restores the number of channels to C ; σ is the Sigmoid activation function, used for normalization. Finally, the output result is obtained by multiplying the above weight values with the original feature matrix to obtain the final output feature vector.

$$\mathbf{X}'_c = \mathbf{X}_c \cdot \mathbf{s}_c (c = 1, 2, \dots, C) \quad (9)$$

Where, $\mathbf{X}'_c$ is the output feature map, $\mathbf{X}_c$ is the input feature map, and $\mathbf{s}_c$ is the weight of the c -th channel.

C. Loss function Design

During convolutional neural network(CNN) training, error values are calculated through loss functions to update parameters and achieve accurate predictions. The loss function of YOLOv8-Pose consists of two main components: the object detection component and the keypoint detection component. The object detection component includes three loss functions: bounding box loss, confidence loss, and category loss. The keypoint detection component comprises both keypoint position loss and confidence loss. This paper improves the bounding box loss function and provides a comprehensive overview of the loss function composition.

(1) Target detection loss

The loss function for object detection measures the discrepancy between the predicted bounding boxes and the actual object boxes. The original network uses CIoU(Complete Intersection over Union) as the calculation standard for regression loss. In the task of pallet detection and localization, the angle of the pallet, as an important component of pose, has a significant impact on the final accuracy. However, CIoU only considers the overlap area and center distance, without considering the consistency of angle.

SIoU (Scaled Intersection over Union)[19] is an improved bounding box regression loss function. Compared with traditional IoU, SIoU optimizes the position, size, shape, and direction of the bounding box through gradient constraints, thereby improving the localization accuracy and convergence speed of object detection. Its core is to construct a more comprehensive loss function by combining the geometric characteristics of the bounding box, so that the optimization process simultaneously focuses on position alignment, shape matching, and direction consistency. SIoU includes three penalty functions, as follows:

$$L_{box} = 1 - IoU + \frac{\Delta + \Omega}{2} \quad (10)$$

Where, IoU is the traditional intersection-union ratio penalty term, Δ is the angular distance penalty term, and Ω is the shape penalty term.

To calculate the angular distance penalty term, the angular function Λ is needed to be calculated, as shown in Figure 5.

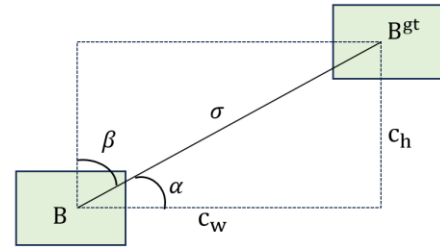


Figure 5. Calculation method of angle penalty term

The calculation formula is as follows:

$$x = \sin(\alpha) = \frac{c_h}{\sigma} = \frac{\max(b_{cy}^{gt}, b_{cy}) - \min(b_{cy}^{gt}, b_{cy})}{\sqrt{(b_{cx}^{gt} - b_{cx})^2 + (b_{cy}^{gt} - b_{cy})^2}} \quad (11)$$

$$\Lambda = 1 - 2 \sin^2 \left(\arcsin(x) - \frac{\pi}{4} \right) \quad (12)$$

The calculation method of the angular distance penalty term is shown in Figure 6. Compared to the distance penalty term of CIoU, the angular distance penalty term of SIoU incorporates a term related to angle. The calculation formula is as follows:

$$\Delta = \sum_{t=x,y} (1 - e^{-\gamma \rho_t}) \quad (13)$$

$$\rho_x = \left(\frac{(b_{cx}^{gt} - b_{cx})^2}{c_w} \right), \rho_y = \left(\frac{(b_{cy}^{gt} - b_{cy})^2}{c_h} \right), \gamma = 2 - \Lambda \quad (14)$$

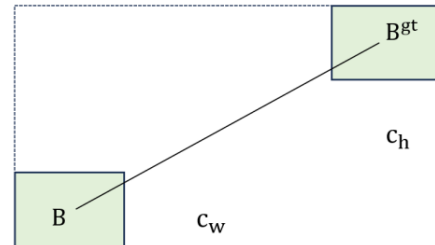


Figure 6. Calculation method of distance between true value bounding box and predicted value

The shape penalty term mainly measures the length and width of the two boxes and their corresponding ratios, calculated as shown in Equation(15). Here, θ is an adjustable parameter used to adjust the proportion of the shape penalty term.

$$\Omega = \sum_{t=w,h} (1 - e^{-\omega_t})^\theta \quad (15)$$

$$\omega_w = \frac{|w-w^{gt}|}{\max(w,w^{gt})}, \omega_h = \frac{|h-h^{gt}|}{\max(h,h^{gt})} \quad (16)$$

The expression for the regular intersection-union ratio penalty term is as follows:

$$\text{IoU} = \frac{|B \cap B^{gt}|}{|B \cup B^{gt}|} \quad (17)$$

Where $|B \cap B^{gt}|$ represents the overlapping area between the predicted box and the ground truth box, and $|B \cup B^{gt}|$ represents the joint area between the predicted box and the ground truth box. Replacing CIoU with SIoU in the YOLOv8 loss function can enhance the network's sensitivity to the angle of the pallet, thereby effectively improving the accuracy of object recognition.

L_{obj} (Objectness Loss, Confidence Loss) measures whether each prediction box contains an object, using BCE (Binary Cross Entropy) Loss or its variant Focal Loss to balance the impact of positive and negative samples:

$$L_{obj} = -\frac{1}{N} \sum_i [y_i \log(p_i) + (1 - y_i) \log(1 - p_i)] \quad (18)$$

Where y_i is the true value and p_i is the predicted value.

L_{cls} (Classification Loss, category loss) is used to measure the difference between the predicted category distribution and the true category distribution using BCE or its variants:

$$L_{cls} = -\frac{1}{N} \sum_i \sum_c y_{i,c} \log(p_{i,c}) \quad (19)$$

Where $y_{i,c}$ and $p_{i,c}$ are the true and predicted class probabilities, respectively.

(2) Keypoint detection loss

Keypoint detection is the core part of YOLOv8-Pose, and its loss function consists of two parts: keypoint position loss and keypoint confidence loss. The keypoint position loss is used to measure the gap between the predicted position of each keypoint and the true position, usually using $\mathcal{L}_1$ Loss:

$$L_{kptl} = \frac{1}{N_k} \sum_i \sum_j \mathcal{L}_1(p_{i,j}, g_{i,j}) \quad (20)$$

Where $p_{i,j}$ is the predicted position of the j -th keypoint for the i -th sample, $g_{i,j}$ is the corresponding ground truth position, and N_k is the total number of keypoints.

The keypoint confidence loss measures whether each keypoint has an object, using BCE loss:

$$L_{kptc} = -\frac{1}{N_k} \sum_i \sum_j [c_{i,j} \log(\hat{c}_{i,j}) + (1 - c_{i,j}) \log(1 - \hat{c}_{i,j})] \quad (21)$$

Where $c_{i,j}$ is the true confidence (1 if there is a keypoint, otherwise 0), and $\hat{c}_{i,j}$ is the predicted confidence.

(3) Total loss

Combine the above losses and balance the tasks of object detection and keypoint detection. The formula for the total loss is:

$$L_{total} = \lambda_{box} L_{box} + \lambda_{obj} L_{obj} + \lambda_{cls} L_{cls} + \lambda_{kptl} L_{kptl} + \lambda_{kptc} L_{kptc} \quad (22)$$

Where $\lambda_{box}, \lambda_{obj}, \lambda_{cls}, \lambda_{kptl}, \lambda_{kptc}$ are weight coefficients used to balance the impact of each part of the loss on the total loss.

D. Adaptive pallet contour extraction

The improved YOLOv8-Pose network is used to detect the target and key points of the pallet in the image, which can initially obtain the position information of the pallet target box and corner points. Generally, the external dimensions of the pallet are relatively fixed, and the corner points identified can be connected in the designed order to extract the contour information of the pallet. However, due to the poor quality of some keypoints, it will have a huge impact on the contour accuracy. Therefore, this paper combines edge detection methods to achieve adaptive contour extraction.

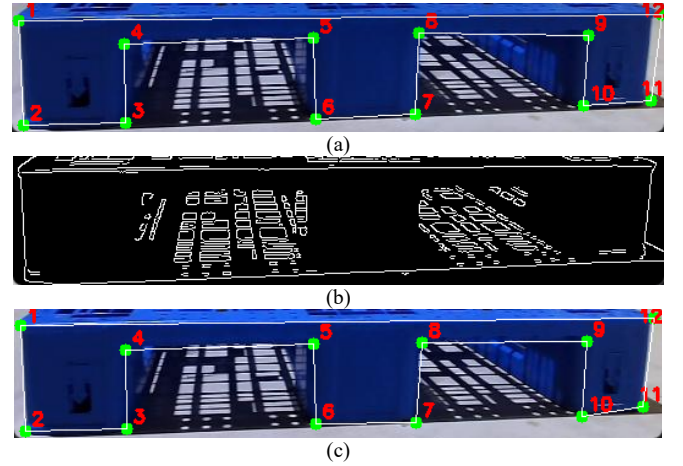


Figure 7. Results of adaptive pallet contour extraction method

The Canny edge detection algorithm[20] is a classic edge detection method with good detection performance and adaptability. The pallet contour extraction method proposed in this paper combines Canny edge detection to achieve adaptive adjustment of edges. Firstly, the pallet area detected by YOLOv8 is cropped, as shown in Figure 7(a). Secondly, the image is converted to a grayscale image, and the edge information of the image is extracted using the Canny edge detection algorithm, as shown in Figure 7(b). According to actual testing, the edge detection parameters G_L and G_H are set to 100 and 200, respectively. Finally, the edge information is matched with keypoints. If the keypoint deviates significantly from the edge, it is adjusted to the nearest edge point, as shown in Figure 7(c). Comparing Figure 7(a) with Figure 7(c), for points 11 and 12 that deviate significantly, the adaptive algorithm proposed in this chapter can effectively improve the contour extraction accuracy.

IV. EXPERIMENTAL DATASET AND EVALUATION METRIX

A. Design of Dataset

Currently the existing pallet detection datasets are all object detection datasets, which cannot meet the training and evaluation requirements of the method proposed in this paper. Therefore, a pallet dataset was designed for object detection and keypoint detection. The dataset was collected using an RGB camera in a warehouse, with videos captured under different background environments, at different times, and from different angles. A total of 1960 frames with high quality and significant differences were selected to form the dataset, as shown in Figure 8. Label the dataset using the data annotation software Labelme. Label the target boxes and anchor points. After labeling, convert the file format to the YOLO format and randomly split the dataset into training, validation, and test sets, with the training set accounting for 80%, the validation set accounting for 10%, and the test set accounting for 10%.

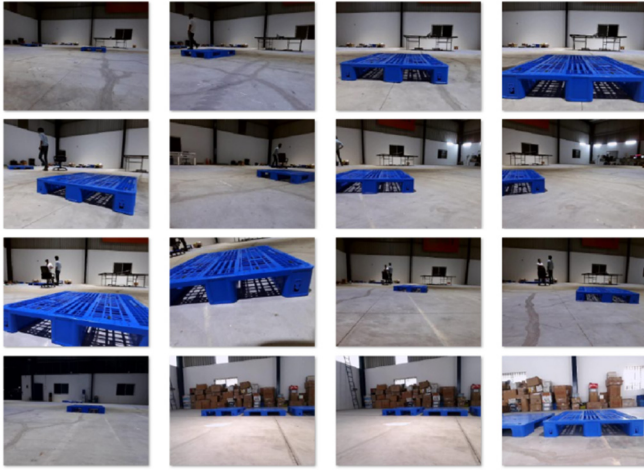


Figure 8. Example images of the pallet dataset

To verify the accuracy of the algorithm proposed in this paper for calculating the pose of the pallet, a verification dataset for determining the angle and position of the pallet pose was further designed. The angle range is from -50° to $+50^\circ$, with a shooting interval of 10° . The distance range is from 1000mm to 3000mm, with a set of images taken every 100mm. In addition, from the pose verification dataset, a dataset of missing image data due to close proximity or large pose deviations was collated to verify the robustness of the pose calculation algorithm. The original images of this dataset are shown in Figure 9.



Figure 9. Images from the dataset of missing image data

B. Design of evaluation metrics

The pallet detection algorithm primarily consists of two components: target detection and center point localization, as well as pallet pose calculation. To thoroughly assess the efficacy of the proposed algorithm, the following evaluation metrics are introduced.

(1) Object detection metrics

The performance of object detection is quantified using AP (average precision). Predicted samples are divided into true positive, false positive, true negative, and false negative based on the relationship between the predicted value and the true value. As shown in Table 1.

Table 1. Introduction to the relationship between predicted values and true values

Concept	Estimate	True value	Is the prediction accurate
True-Positive, TP	Positive sample	Positive sample	correct
False-Positive, FP	Positive sample	Negative sample	error
True-Negative, TN	Negative sample	Negative sample	correct
False-Negative, FN	Negative sample	Positive sample	error

The accuracy rate represents the proportion of correctly predicted samples among all predicted positive samples. The specific formula is as follows:

$$Precision = \frac{TP}{TP+FP} \quad (23)$$

The recall rate represents the proportion of correctly predicted samples among all true positive samples. The specific formula is as follows:

$$Recall = \frac{TP}{TP+FN} \quad (24)$$

Using a reference value to classify samples, we predict that images with IoU values greater than the reference value are positive samples, and those with values less than the reference value are negative samples. By dividing the samples with different reference values, we calculate the accuracy and recall rate corresponding to different reference values, and plot a PR curve with the recall rate on the horizontal axis and the accuracy on the vertical axis. The AP value is given by calculating the area under the PR curve.

(2) Evaluation metrix of keypoint detection

The AP for keypoints (AP^{kp}) is a crucial metric used to evaluate the performance of models in the field of human keypoint detection. Initially, the Object Keypoint Similarity (OKS) is computed. OKS assesses the similarity between detected keypoints and their corresponding ground truths using a weighted average, taking into account both the significance and scale information. Its calculation formula is as follows:

$$OKS = \frac{\sum_i \exp\left(-\frac{d_i^2}{2s^2k_i^2}\right)\delta(v_i>0)}{\sum_i \delta(v_i>0)} \quad (25)$$

Where d_i is the Euclidean distance between the i -th keypoint and the ground truth point. s is the scale factor of the target instance, which is replaced by the area of the bounding box. v_i is the visibility flag of the keypoint, with a value of 0 indicating that the point does not exist in the prediction, 1 indicating that the point exists but is occluded, and 2 indicating that the keypoint exists and is visible. k_i is the weight constant of the i -th keypoint, which reflects the importance of different keypoints. Δ is an indicator function, which is 1 when the keypoint is visible and 0 otherwise.

The closer the value OKS is to 1, the smaller the error of the keypoint. The closer OKS is to 0, the worse the detection effect. Similar to the AP index, the average accuracy AP^{kp} can be calculated based on the OKS under the PR curve.

V. EXPERIMENTAL RESULTS AND ANALYSIS DISCUSSION

This paper presents an improved algorithm developed on the NVIDIA GeForce RTX 4090 GPU using the PyTorch framework. To prevent network overfitting, data augmentation techniques are implemented: random chromaticity space adjustments during training and image flipping with a 50% probability threshold. Additionally, Mosaic data augmentation is applied by randomly cropping and stitching four images for training, effectively enriching background diversity and increasing batch sizes. The enhanced YOLOv8-Pose algorithm was trained on self-made pallet dataset with a learning rate of 0.01. Target detection metrics and keypoint detection metrics were calculated on the validation set, achieving optimal performance after 1200 iterations.

A. Object detection and keypoint detection results

Object detection and keypoint detection are the basis for pallet pose calculation. Object detection provides a cropping framework for image processing. To verify the effectiveness of the proposed method for object detection, the method was validated on a pallet dataset. Figure 10 shows some results of the pallet recognition using the proposed method.

The results of the improved YOLOv8-Pose algorithm were compared with those of YOLOv8s, and further experiments were conducted using other popular object detection algorithms. The results are shown in Table II. According to Table 2, the proposed method achieved an average accuracy improvement of 1.2% over the original network YOLOv8s, reaching 93.3%; the recall rate increased by 0.3%, reaching 98.3%; and it significantly improved compared to classic detection networks such as Cascade R-CNN[21], CenterNet[22], YOLOv5, and YOLOv7. In terms of frame rate, the proposed method slightly decreased compared to the original network frame rate, but the decrease was not more than 5%, and 117.7 frames were fully capable of meeting the needs of trajectory tracking.

Keypoint detection is the premise of subsequent contour extraction. The results of the improved YOLOv8-Pose method are compared with other methods to verify the effectiveness of keypoint detection. The results are shown in Table 3. Compared with the YOLOv8-Pose algorithm, the proposed method achieves a 3.7% AP^{kp} improvement, AP_{50}^{kp} and AP_{75}^{kp} with improvements of 1.2% and 4.3% respectively. Compared with the YOLOv7-Pose algorithm, the AP^{kp} improvement is as high as 10.1%. The above comparison proves the effective role of the improvement for keypoint detection tasks.



Figure 10. Results of pallet target detection using the proposed method

Table 2 Comparison of target detection results on pallets

Methods	AP (%)	Recall (%)	FPS
Cascade R-CNN[21]	81.4	88.3	12.1
CenterNet[22]	87.3	93.1	42.2
YOLOv5s	88.7	97.1	148.1
YOLOv7-w6	91.1	97.3	78.6
YOLOv8s	92.1	97.9	121.2
Proposed method	93.3	98.3	117.7

Table 3 Comparison of detection results of key points on pallets

Methods	AP_{50}^{kp} (%)	AP_{75}^{kp} (%)	AP^{kp} (%)
YOLOv7-Pose	91.3	67.1	64.7
YOLOv8-Pose	93.9	73.2	71.1
Proposed method	95.1	77.5	74.8

B. Experiment of keypoint missing image

When the relative distance between forklift and pallets is close or the pose deviation is large, the deformation of the pallet image is more severe, posing higher requirements on network performance. In addition, due to the inability to obtain complete keypoint information, existing algorithms cannot directly calculate the pose of the pallet. Therefore, the algorithm is validated on a keypoint missing image dataset proposed in this paper to demonstrate its robustness.

The results of the proposed method are compared with the YOLOv8-Pose. Figure 11 shows the comparison of keypoint detection for some images. The keypoints of the proposed method are more complete, with fewer keypoint offsets, showing significant improvement compared to YOLOv8-Pose.

The statistical results are presented in Table 4. For the image dataset lacking keypoint data, the proposed method outperforms the original YOLOv8-Pose network, with an AP increase of 0.9% to 92.5%, a recall rate increase of 0.6% to 97.9%, and keypoint metrics AP^{kp} and AP_{50}^{kp} improvements of 5.7% and 3.6%, respectively. Evidently, the enhancements made to the YOLOv8 network proposed in this paper are particularly effective in scenarios involving severe object deformation and missing keypoints.

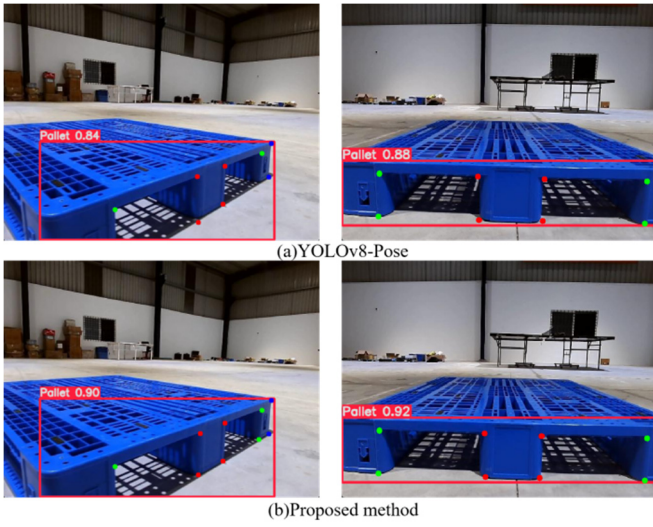


Figure 11. Comparison of keypoint detection using YOLOv8-Pose and the proposed method

Table 4 Experimental results of images with missing keypoints

Metric	YOLOv8-Pose	Proposed method
AP (%)	91.6	92.5
Recall (%)	97.3	97.9
AP ^{kp} (%)	64.1	69.8
AP ₅₀ ^{kp} (%)	87.9	91.5

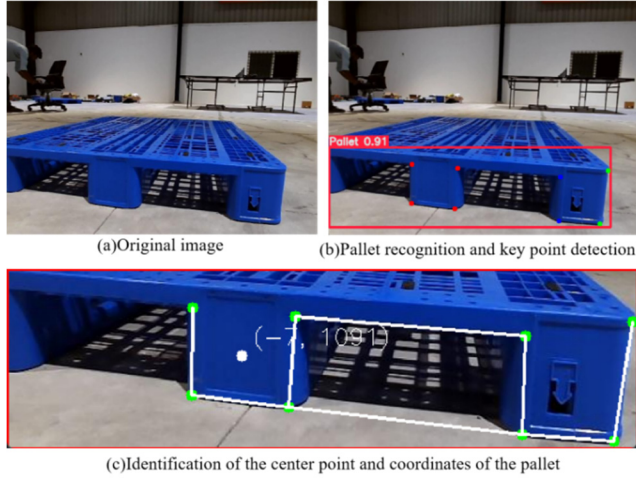


Figure 12. Calculation process of image pose with missing keypoints

When keypoints are missing, it cannot obtain center point information by connecting keypoints directly. Through the proposed adaptive pallet pose calculation method, flexible selection of calculation point pairs is achieved, thus enabling pallet pose calculation for such images. As shown in Figure 12, firstly, target recognition and keypoint recognition are performed on the original image. Then, pallet contour extraction and pallet pose calculation method based on geometric relationships are applied. Finally, recognition results with pallet center points and center coordinates are obtained.

C. Real-world environmental validation

To comprehensively evaluate the effectiveness of intelligent pallet picking by forklifts proposed in this paper, multiple sets of repeated experiments were conducted under different interference environments, targeting different pallet positions and different pallet inclinations. The CBD20/AGV forklift AGV from Ningbo Ruyi Co., Ltd. Was used. The forklift is a small-sized handling forklift, typically used in small and medium-sized warehouses for pallet handling. The final experimental platform of unmanned forklift AGV is shown in Figure 13. As the proposed method only requires a monocular camera to achieve the algorithm information collection, the requirements for the camera are relatively low. The BlueCore EAGLE series LXPS-DS4423 camera was used, and the camera was installed on the front of the forklift through a customized frame. The host computer was a JPB13 portable computer from Shenzhen Zhongbai Technology, and the industrial control computer was fixed on the upper part of the forklift. The controller was integrated inside the forklift.

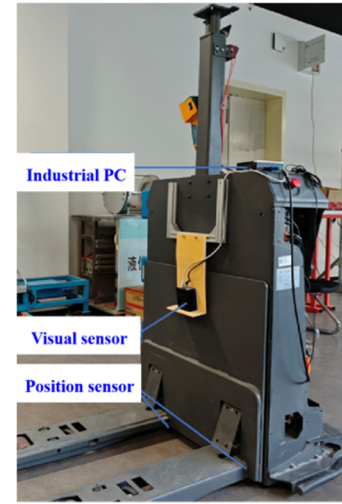


Figure 13. Schematic diagram of experimental platform

The test was conducted in a non-interfering environment, with tests performed at distances of 3m, 5m and 7m. Given that the tilt angle of the pallet is typically not excessively large in practical scenarios, tasks were set up with initial tilt angles of 0°, 10°, 20° and 30° for each distance, with 15 tests conducted for each condition. The test results are presented in Table 5. By organizing and summarizing the results, Table 5 and Table 6 were derived. At a distance of 3m, the success rate was the lowest due to limited adjustment space caused by the close proximity. As the initial tilt angle of the pallet increased, the path became more convoluted, leading to a decrease in the success rate of pickup. In non-interfering conditions, the overall success rate of the proposed method in this paper reached 93.8%. Even when the tilt angle of the pallet reached 30°, the comprehensive success rate remained above 85%, satisfying engineering requirements. The process of pallet forklift pickup is illustrated in Figure 14.

Table 5 Statistics of Pallet picking success rate without interference

Distance (m)	Pallet (°)	inclination number successes	Success rate
3	0	14	93.3%
3	10	14	93.3%

3	20	13	86.7%
3	30	12	80%
5	0	15	100%
5	10	15	100%
5	20	15	100%
5	30	14	93.3%
7	0	15	100%
7	10	15	100%
7	20	14	93.3%
7	30	13	86.7%

Table 6 Analysis of Pallet picking success rate without interference

Parameter	Distance (m)			Pallet inclination (°)			
	3	5	7	0	10	20	30
parameter value	3	5	7	0	10	20	30
number of successes	53	59	57	44	44	42	39
total count	60	60	60	45	45	45	45
Success rate	88.3%	98.3%	95.0%	97.7%	97.7%	93.3%	86.6%



Figure 14. Example of pallet picking process using unmanned forklift AGV

VI. CONCLUSIONS

A monocular pallet recognition algorithm based on improved YOLOv8-Pose is proposed in this paper. Firstly, the algorithm is improved by replacing part of C2f with C2f_DCN to adapt to geometric changes of pallets; the SEAttention module is added to reduce the influence of irrelevant features and improve network performance; the SIoU loss is introduced considering the angle information of pallets to improve training accuracy. Secondly, based on obtaining the target box and keypoint information, Canny edge detection is used to adaptively adjust edge features to obtain more accurate contour information. Finally, the self-made pallet dataset and actual environment are used for verification. The experimental results demonstrate that the proposed method is superior to the YOLOv8s algorithm in target detection and keypoint detection, with an average accuracy of 93.3%; a recall rate of 98.3%, and an average keypoint accuracy improvement of 5.7%.

ACKNOWLEDGMENT

This work is supported by the Project of Major Science and Technology Research Project in Ningbo of China (Grant No.2023Z041, No.2023Z027 and 2024Z165).

REFERENCES

- [1] Lecking D, Wulf O, Wagner B. Variable pallet pick-up for automatic guided vehicles in industrial environments[C]. IEEE Conference on Emerging Technologies & Factory Automation, 2006:1169-1174.
- [2] Jo M, Youn H Y, Chen H H. Intelligent RFID tag detection using support vector machine[J]. IEEE Transactions on Wireless Communications, 2009, 8(10):5050-5059.
- [3] Wang Mingjun. Visual-based warehouse pallet detection and adaptive pose stabilization for mobile robots[D]. Hangzhou Dianzi University, 2022.
- [4] Mok C, Baek I, Cho Y S, et al. Pallet recognition with multi-task learning for automated guided vehicles[J]. Applied Sciences, 2021, 11(24): 11808.
- [5] Walter M R, Karaman S, Frazzoli E, et al. Closed-loop pallet manipulation in unstructured environments[C]. IEEE/RSJ International Conference on Intelligent Robots and Systems, 2010: 5119-5126.
- [6] Wang S, Ye A, Guo H, et al. Autonomous pallet localization and picking for industrial forklifts based on the line structured light[C]. IEEE International Conference on Mechatronics and Automation, 2016:707-713.
- [7] Xiao J, Lu H, Zhang L, et al. Pallet recognition and localization using an RGB-D camera[J]. International Journal of Advanced Robotic Systems, 2017, 14(6):172988141773779.
- [8] Chen Qian. Implementation of a Pallet Picking Scheme Based on Monocular Vision Guidance[D]. Sichuan University, 2021.
- [9] Ye Wei. Research and Implementation of Vision-Based Navigation and Pallet Positioning Methods for Unmanned Forklifts[D]. China University of Science and Technology, 2021.
- [10] Zhu Daping, Zhu Ming, and Zhou Hengsen. An improved CenterNet-based pallet localization method [J]. Computer System Applications, 2022,31(10):303-309.
- [11] Tsiogas E, Kleitsiotis I, Kostavelis I, et al. Pallet detection and docking strategy for autonomous pallet truck AGV operation[C]. IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), 2021: 3444-3451.
- [12] Liu Jizhe. Lightweight pallet detection algorithm design and implementation for industrial mobile robots[D]. University of Electronic Science and Technology of China, 2024.
- [13] Zhou Hengsen. Research on precise positioning and road detection of pallets in visual navigation unmanned forklifts [D]. China University of Science and Technology, 2023.
- [14] Redmon J. You only look once: Unified, real-time object detection[C]. IEEE conference on computer vision and pattern recognition, 2016.
- [15] Maji D, Nagori S, Mathew M, et al. Yolo-pose: Enhancing yolo for multi person pose estimation using object keypoint similarity loss[C]. IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2022: 2637-2646.
- [16] Dai J, Qi H, Xiong Y, et al. Deformable convolutional networks[C]. IEEE international conference on computer vision, 2017: 764-773.
- [17] Xueli Shen and Lingchao Wang. UAV aerial target detection based on YOLOv8n [J]. Computer Systems & Applications, 2024,33(7):139-148.
- [18] Hu J, Shen L, Sun G. Squeeze-and-excitation networks[C]. IEEE conference on computer vision and pattern recognition, 2018: 7132-7141.
- [19] Gevorgyan Z. SIoU loss: More powerful learning for bounding box regression[J]. arXiv preprint arXiv:2205.12740, 2022.
- [20] Ding L, Goshtasby A. On the Canny edge detector[J]. Pattern recognition, 2001, 34(3): 721-725.
- [21] Cai Z, Vasconcelos N. Cascade r-cnn: Delving into high quality object detection[C]. IEEE conference on computer vision and pattern recognition, 2018: 6154-6162.
- [22] Zhou X, Wang D, Krähenbühl P. Objects as points[J]. arXiv preprint arXiv:1904.07850, 2019.